生产过程中最优决策问题的研究

摘要

企业在生产电子产品的过程中会面临一些决策问题，而决策出的方案会影响成本，进而影响利润。本文中我们主要在零件合格率及组装成功率一定时，是否检测零件、是否检测和拆解半成品和成品进行了讨论，并给出较优的决策方案。当零件合格率未知时，可通过一些办法预估次品率带入模型求解最优策略。

对于问题一, 本文建立了**二项分布模型**及**正态分布模型**进行求解。首先，将零件的合格与否转化为二项分布中的 1 和 0，然后在零件数足够多的前提下，根据中心极限定理，对该模型进行合理近似，使其成为正态分布模型，进而通过 Cochrane 公式对最小样本空间进行了求解。经过计算，我们得出结论：在情形一下，**至少需要抽样检测 139 个零件，如果其中有至少 20 个不合格，则拒收；在情形二下，至少需要检测 98 个零件，若其中有至少 84 个合格，则接收**。

对于问题二, 本文采用**非线性整数规划模型**进行求解。将目标函数设为某种决策方案获得的利润，通过分步计算成本和售价，将目标函数表示为决策变量的非线性形式。限制条件要求决策变量为 0-1 变量，对该规划问题进行求解，我们在各种情况下均得到了合理的决策方案，具体求解结果见正文。

对于问题三, 本文在问题二的基础上，采用**贪心算法**进行求解，并使用**遗传算法**进行优化。我们将问题三中的工序进行拆分，每一道工序可视为几个零件组装为一个成品，我们从而可以利用问题二中的模型得到该工序下的最优解。由该工序下的最优决策方案可计算出成品的次品率等数据，用于对下一道工序进行决策。重复这个过程，我们得到了一个局部最优解。同时，我们可以通过改变限制条件得到一些较优解，构成可行解域，通过遗传算法的交叉变异过程，优化该局部最优解。在问题三给出的特定情况下，我们利用上述模型求解得到的最优方案为：**对零件一、二、四、五、七、八进行检测，对所有半成品合成品进行检测和拆解，对应利润为 11516.2 元**。

对于问题四, 本文利用**假设检验**的方式确定次品率，后利用问题二、三中已有模型，给出在次品率不确定时的决策方法。我们首先进行小规模的抽样检测，得到初步次品率，再根据问题一中的方式确定对应的最小样本空间及容错范围，检验该次品率的可靠性。在可靠性检验失败的情况下重新重复上述过程。由于我们排除异常了数据，所以最终一定会得到一个较为准确的次品率，可将其近似为实际次品率进行求解，进而得出最优决策方案。

关键字：市场决策　贪心算法　遗传算法　假设检验　非线性整数规划

一、 问题重述

1.1 问题背景

企业在生产电子产品时，往往需要从供应商处购买零件进行组装，将零件逐步加工成半成品，再加工成成品。而在进货时，部分零部件可能出现损坏，而用损坏的零部件组装而成的半成品和成品一定不合格。不合格的产品流入市场，会对企业的声誉造成影响，还会造成赔偿和交通损失。为避免这种情况的发生，企业可以对零部件，半成品和成品进行检测，丢弃或拆解其中不合格者，但检测本身有一定成本，拆解亦然。于是，是否进行检测，是否进行拆解便构成了一个决策问题，需要我们建立模型进行量化分析。

图 1　决策问题示意图

1.2 问题要求

问题 1 该问题要求我们对于次品率有大概了解的情况下设计一个检测方案，对供应商提供的零部件进行检测。对于这个方案而言，要求检测数量尽量少从而最大降低检测时产生的成本，且能达到一定信度，能够验证预先了解到的次品率值。具体考虑以下情形：

1. 信度为 95%，若次品率超过 10%，则拒收。
2. 信度为 90%，若次品率不超过 10%，则接收。

问题 2 该问题要求我们针对生产的三个阶段给出决策方案，具体为：

1. 是否检测零部件
2. 是否检测半成品
3. 是否拆解成品

对于不同的决策情形，对成本产生的影响不尽相同。而且对零部件检测相互之间也会产生利润上的影响，问题较为复杂，需要考虑众多因素。因此需要综合分析各种决策对于成本的影响，结合多情况给出最优决策。

问题 3 该问题在问题二的基础上进行了拓展，要求我们考虑多个零部件和多道工序的情况，最开始时依据题意，以 8 个零件，2 道工序为例，给出决策方案。之后拓展结果，考虑 n 个零配件，m 道工序的情况，由特殊到一般推广结论。

问题 4 该问题对次品率给出了新的定义：通过抽样检测计算得到的、数据不完全可靠的次品率。并要求我们在此情境下给出决策方案。

二、 问题分析

2.1 问题一分析

对于问题一，题目要求我们根据抽样调查的结果来判断是否接收这批零件。注意到单个零件只有好坏两个状态，可用 0 和 1 分别表示。在样本空间较大的情况下，零部件的好坏情况总体上服从二项分布。鉴于实际情况中零件数足够多，可近似认为其服从正态分布，由此，我们便可利用相关统计学公式对抽样检测样本量的最小值进行求解。

2.2 问题二分析

对于问题二，题目要求我们针对多种情况做出决策，以达到成本尽量低，利润尽量高的效果。具体需要我们判断三个步骤，即是否检测零件，是否检测成品以及是否拆解不合格成品。

这实际上是一个决策问题，因此我们建立一个非线性整数规划的数学模型，以成本水平的大小为目标函数，约束条件选择使用求解变量都为 (0-1) 变量这一事实，综合考虑各种方案的带来的收益，做出最优决策。指导商品生产。

在这个模型中，我们已知成品零配件的组成和各种环节次品率的大小，通过更改一些模型中的参数，就可以得到题设中各种情况的对应的最优决策。

2.3 问题三分析

对于问题三，通过观察，我们发现其本质是对问题二的拓展，需要我们对问题二的模型进行改进，使得新得到的模型能够适用于多个零件和多道工序的决策。为此，我们在问题二的基础上使用了贪心算法，分步解决问题，最大化降低成本，提高产品利润。

我们首先将问题三划分为两个步骤：半成品的组装和成品的组装。对于半成品的组装，其方法同问题二。计算出该步骤的最优解后，我们便可以考虑半成品的次品率，将其视作零件，那么其又可以视作问题二来解决。最终，我们便可以得到最优解。

2.4 问题四分析

对于问题四，需要综合先前问题中的相关结论。利用问题一的方法，根据样本空间中的次品数目来判断次品率的上下限。最后利用问题三中的模型进行求解，得到最优检测方案。

由于预先不清楚实际次品率，所以需要提前对次品率进行评估。在此指定一个较小的样本空间，例如 50，检测这些样本得到初步次品率，以初步次品率为依据带入问题一中的结论计算出最小样本空间，并将其补全。

接着重复问题一中的可靠性检验，如果检验合格就认为初步次品率可靠，反之，将最初的样本排除，在新得到的样本中抽样得到新的初步次品率。重复先前步骤，直到检验合格。

三、 模型假设

为简化问题，本文做出以下假设：

- 假设 1 问题一中的样本量足够大，近似服从正态分布
- 假设 2 仅考虑题中会对产品成本造成影响的因素，不考虑其他小概率因素
- 假设 3 抽样检测结果一定准确，且没有检测失败的可能
- 假设 4 所有的成品会被购买，不存在成品滞销的情况

四、 符号说明

符号	说明	单位
$x_{1\sim4}$	决策变量	-
p_1	零件一合格概率	-
p_2	零件二合格概率	-
p_3	成品装配合格概率	-
p	成品合格概率	-
c_1	零件一成本	元
c_2	零件二成本	元
c_3	装配成本	元
t_1	零件一检测单价	元
t_2	零件二检测单价	元
t_3	成品检测单价	元
t_4	成品退换单价	元
n_1	零件一数量	-
n_2	零件二数量	-
n_3	卖出成品数量	-
$\bar{n_3}$	次品数量	-
s	成品单价	元
W	利润	元
Z	成本	元

五、 问题一的模型的建立和求解

5.1 模型建立

5.1.1 二项分布模型的建立

考虑单个零件，发现其只有两种状态：合格和不合格。我们用 1 表示零件合格，0 表示零件不合格，p 表示预期零件合格的概率，即 1-表称值，在题目中为 0.9。据此建立二项分布模型模拟这批零件的合格数量情况。即：

$$\mathcal{X} \sim Bin\,(n, p)$$

其中 n 表示检测零件的数量，为待求值。

5.1.2 正态分布模型的近似

在 n 比较大时，根据中心极限定理，在 n 足够大时，二项分布可近似为正态分布，通常情况下，n 达到 100 左右时，该近似的效果可以满足我们的需求。此处，我们不妨假设 n 满足该条件，对上述二项分布进行近似，得到一个正态分布模型：

$$\mathcal{Y} \sim N\left(\mu, \sigma^2\right)$$

其中，μ 表示正态分布的均值，σ^2 表示方差。这两个数据与二项分布相同，可利用二项分布的方法进行计算。有了这些模型，我们便可利用相关公式对最小样本量 n 进行求解。

5.2 模型求解

Step1：二项分布相关数据的计算

为了给出具体的正态分布模型，我们需要对 μ 和 σ^2 进行计算，具体如下：

$$\mu = np = 0.9n$$

$$\sigma^2 = np(1-p) = 0.09n$$

Step2：Z 值的确定

为根据信度估计样本量 n 的大小，我们需要根据题目的要求查找具体信度的 Z 值，通过查询 Z 表，我们得到在 95% 信度下对应 Z 值为 $Z_{0.025} = 1.96$，在 90% 信度下 Z 值为 $Z_{0.05} = 1.645$。

Step3：n 的计算

根据 Cochrane 公式，有

$$n = \frac{Z^2 p(1-p)}{E^2}$$

其中 E 为能容许的最大误差，通常取 0.05。

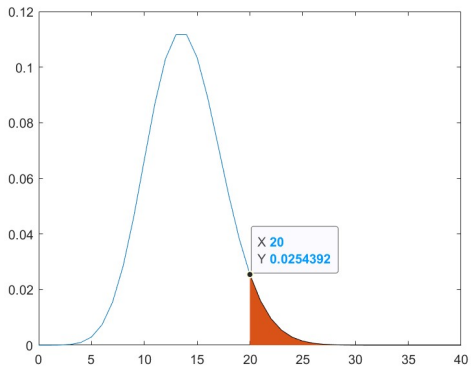
至此我们估算出了最小样本空间 n，将其分别代入两种情形以求解次品数目的临界值。根据正态分布单侧临界点的相关性质，有

$$n_{max} \leq np + Z\alpha\sqrt{Np(1-p)}$$

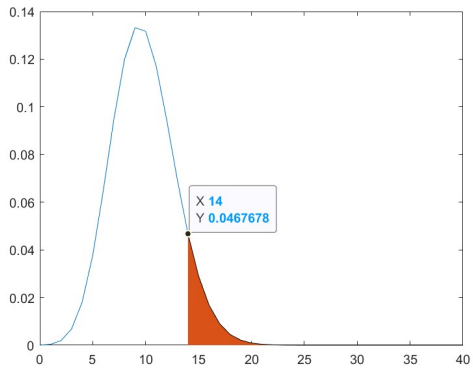
以此为依据得到后续结果。

5.3 求解结果

通过计算，我们得到在标称值为 10% 的前提下，对于情形一，我们至少需要检测 139 个零件，对于情形二，我们至少需要检测 98 个零件。下图为两种情形对应的正态分布图，可以看到，在情形一中，若抽样调查的 139 个零件中有至少 20 个不合格，则拒收。在情形二中，若抽样调查的 98 个零件中有至少 84 个合格，则接收。



((a)) 情形一正态分布图



((b)) 情形二正态分布图

图 2　两种情形的图示

六、 问题二的模型的建立和求解

6.1 模型建立和求解

问题二是一个规划问题，要求我们在多个限制条件下求解成本最低的方案。为此，我们建立规划模型进行解决。

6.1.1 决策函数的建立

注意到，在实际的生产投放中涉及四个决策，我们以此建立四个函数进行表示：

$$x_i = \begin{cases} 1, & \text{执行该种决策} \\ 0, & \text{不执行该种决策} \end{cases}, \quad i = 1, 2, 3, 4$$

其中 x_1、x_2、x_3、x_4 分别对应零件 1 的检测、零件 2 的检测、成品的检测以及成品的拆解。

6.1.2 不合理情况的排除

- 在两款零件不同时检测的情况下，如果我们当我们不进行成品检测时，就一定不会拆解成品，因为我们需要防止次品零件流入组装循环。

- 在两款零件都不检测的前提下，就算进行成品检测，我们也不会拆解检测出的次品零件，因为就算拆解了零件，次品零件还是会重复组装。

6.1.3 成本的计算

题目中的生产过程分为四个阶段，对应地，我们在计算成本时也可按该步骤执行。

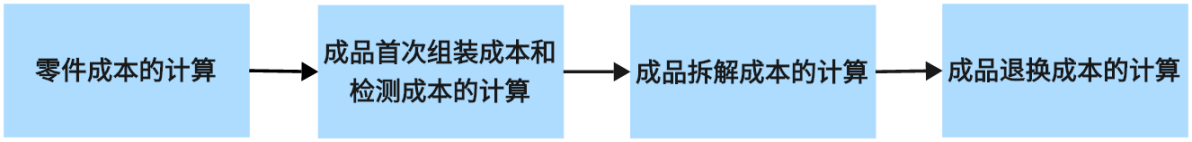


图 3　解决步骤

步骤一：零件成本的计算

记零件 1,2 的成本分别为 c_1 和 c_2，进货数量为 n_1 和 n_2，则固定成本为 $c_1n_1+c_2n_2$，此外，如果进行检测，还会产生检测成本，记单个零件的检测成本为 t_1 和 t_2。因此，零件总成本为：

$$Z_1 = c_1n_1 + c_2n_2 + x_1n_1t_1 + x_2n_2t_2$$

步骤二：成品首次组装成本和检测成本的计算

记组装成本单价为 c_3，则总组装成本为 c_3n_3, 其中 n_3 表示第一次组装成品的数量，为 min{(1-x_1p_1)n_1,(1-x_2p_2)n_2}。此外，还需考虑检测成本，记检测成本的单价为 t_3，则该步骤总成本为

$$Z_2 = c_3n_3 + t_3n_3x_3 = n_3(c_3 + t_3x_3)$$

步骤三：成品拆解成本的计算

该步骤无固定成本，进行拆解时会产生成本，记拆解成本的单价为 t_4, 拆解的数量即为不合格成品的数量，为此计算次品率。考虑到零件数量极为庞大，次品率可由下式近似计算：

$$\bar{p} = 1 - (1 - (1 - x_1)(1 - p_1))(1 - (1 - x_2)(1 - p_2))p_3,$$

考虑到不合格成品的零件拆解后会不断重复上述步骤，实际拆解的数量应为

$$\bar{n_3} = \frac{n_3p}{1-p}$$

则拆解成本为

$$\bar{n_3}t_4 = \frac{n_3p}{1-p}t_4$$

此外，考虑到在执行该决策时在迭代过程中会产生额外的成本，在该步骤一并进行计算。

首先考虑额外检测费用，考虑到实际情况中零件的进货量极大，忽略某种零件不足导致无法装配的问题，则每种零件检测数应和不合格产品数相同，即为 $\bar{n_3}$，那么额外检测成本为

$$\bar{n_3}(x_1t_1 + x_2t_2)$$

然后考虑额外装配费用，与考虑额外检测费用时同理，装配数量应为 $\bar{n_3}$, 那么额外装配费用应为

$$\bar{n_3}c_3$$

综上，该步骤的总成本为

$$Z_3 = \bar{n_3}(t_4 + x_1t_1 + x_2t_2 + c_3)$$

步骤四：成品退换成本的计算

总退换成本即用成品数乘以次品率得到次品数，再与退换单价相乘，为：

$$Z_4 = (1 - x_3)n_3\bar{p}t_4$$

最终成本确定为

$$Z = Z_1 + Z_2 + Z_3 + Z_4$$

6.1.4 利润的计算

对于利润的计算，我们采用卖出成品收益减去成本的方式，公式如下：

$$W = sn_3(1-p)^{1-x_4}min\{1-p_1, 1-p_2\}^{x_4} - Z$$

其中 s 代表成品的售价。

由此，我们可以计算出成本和利润，再取成本小而利润大者，作为最优方案。

6.2 求解结果

利用 matlab 进行求解结果如下：

	是否检测零件一	是否检测零件二	是否检测成品	是否拆解次品
情况一	是	否	是	是
情况二	是	否	是	是
情况三	是	否	是	是
情况四	是	是	是	是
情况五	否	是	是	是
情况六	否	否	否	否

可以看到，得到的结果在零件次品率低，检测成本高时，我们趋向于不检测零件，反之则趋向于检测零件，成品亦然。在拆解费用高于购买零件总价时，我们趋向于不拆解次品，反之则倾向于拆解次品，与我们的常识相符。作为参照，我们也给出了不同情况下各种策略的利润：

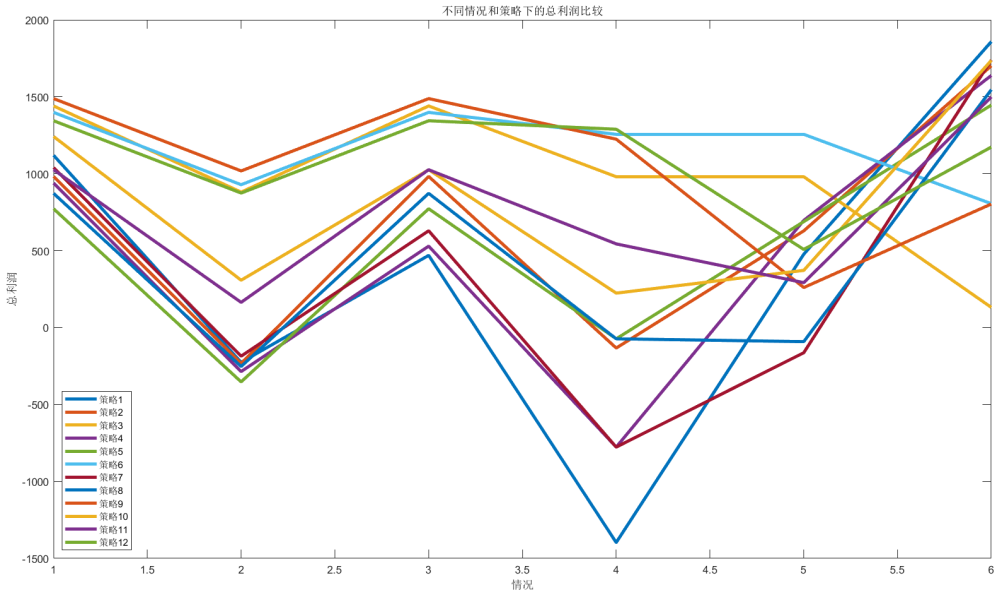


图 4　各种情况下不同策略的收益

七、 问题三的模型的建立和求解

我们在问题二的基础上进行一定程度的修正，然后结合贪心算法进行求解。

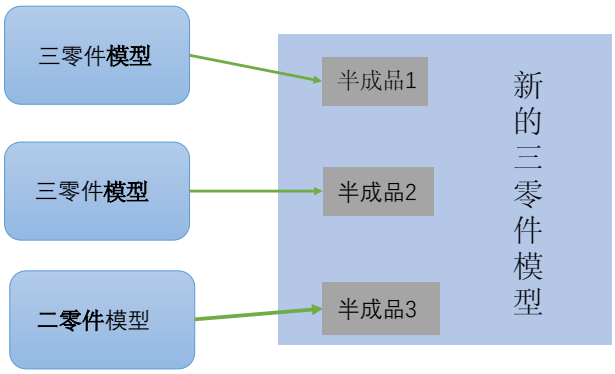


图 5　问题三模型的示意图

7.1 模型建立和求解

在问题三中，零件被首先加工为半成品，再加工成成品。由于直接考虑全过程计算量过大，我们选择逐步讨论，首先考虑零件加工成半成品的最优决策。对比问题二，我们发现二者在一些方面有差异：成品有退换成本和售价，而半成品无直接的退换成本和售价，而是通过影响成品的合格率来间接的增加成本和产生价值。半成品无购买单价，而是通过购买零件进行组装得到。因此，我们需要对半成品的相关定义进行补充。

7.1.1 半成品退换成本的定义

在半成品的数量为 m，成品的退换成本为 t 时，我们定义每个半成品的等效退换成本为 $\frac{t}{m}$，以表示半成品均摊成品中次品流入市场的风险。

7.1.2 半成品售价的定义

类似地，在半成品的数量为 m，成品的售价为 s 时，我们定义每个半成品的售价为 $\frac{s}{m}$，以表示半成品平分成品产生的价值。

7.1.3 半成品成本的定义

对于一个需要 n 个零件组装而成的半成品，设每个零件的成本依次为 c_1,c_2,$\cdots$,c_n，装配成本为 c，则半成品的成本定义为

$$C = c + \sum_{k=1}^{n} c_k$$

给出上述定义后，我们可以采用问题二中的模型进行求解。在问题三中，三个半成品合成一个成品，我们可以类似的给出成本——利润计算模型。在该模型下可以计算出半成品的次品率，然后半成品到成品的过程便又可以使用问题二的模型进行求解，最终得到最优方案。鉴于贪心算法得到的只是局部最优解，我们可以使用遗传算法进行优化。

7.1.4 优化模型的建立和求解

Step1：模型原理的说明

遗传算法模拟的是生物进化的过程，是一种基于自然选择原理和自然遗传机制的搜索算法。其核心步骤如下：

1. 确定可行解域及编码方法，在本题中，共有 12 个决策变量，对应可行解域共 2^{12} 个元素，分别表示是否检测零件，半成品和成品。编码方法采用二进制，检测为 1，不检测为 0。则可行解域的每一解均可用长为 12 的字符串表示。
2. 建立适应度函数确定度量好坏的依据。在本题中即为利润大小，利润越大，适应度越高，所得解越接近最优解。
3. 确定进化参数。在本题中，我们只考虑最优解的变异，即将某一步骤的最优解替换为较优解。
4. 确定选择策略。计算变异后的最优解，与之前比较，选择总利润更高者作为新解。

Step2：模型的具体应用

根据贪心算法，我们将该问题分解为了 4 个模块，模块 1 为零件 1~3 合成半成品 1 的步骤，模块 2 为零件 4~6 合成半成品 2 的步骤，模块 3 为零件 7~8 合成半成品 3 的步骤，模块 4 为半成品合成成品的步骤。模块 1~3 中策略的选择会影响模块 4 的最优解。而模块 4 的最优解即为适应度函数的最大值，因此模块 4 不需要进行变异，只需要根据模块 1~3 的决策确定该种情况下的最优解即可。由此，遗传算法的应用主体为前三个模块。

我们可以通过问题二中模型计算出每个模块每种策略对应的成本和次品率，并得到局部最优解。我们设置变异的范围不低于最优解的利润的 95%，次品率不高于最优解的 110%，以此为条件考虑变异后种群的最优解。

通过这种方法的优化，我们使得由贪心算法得到的局部最优解更加接近全局最优解。

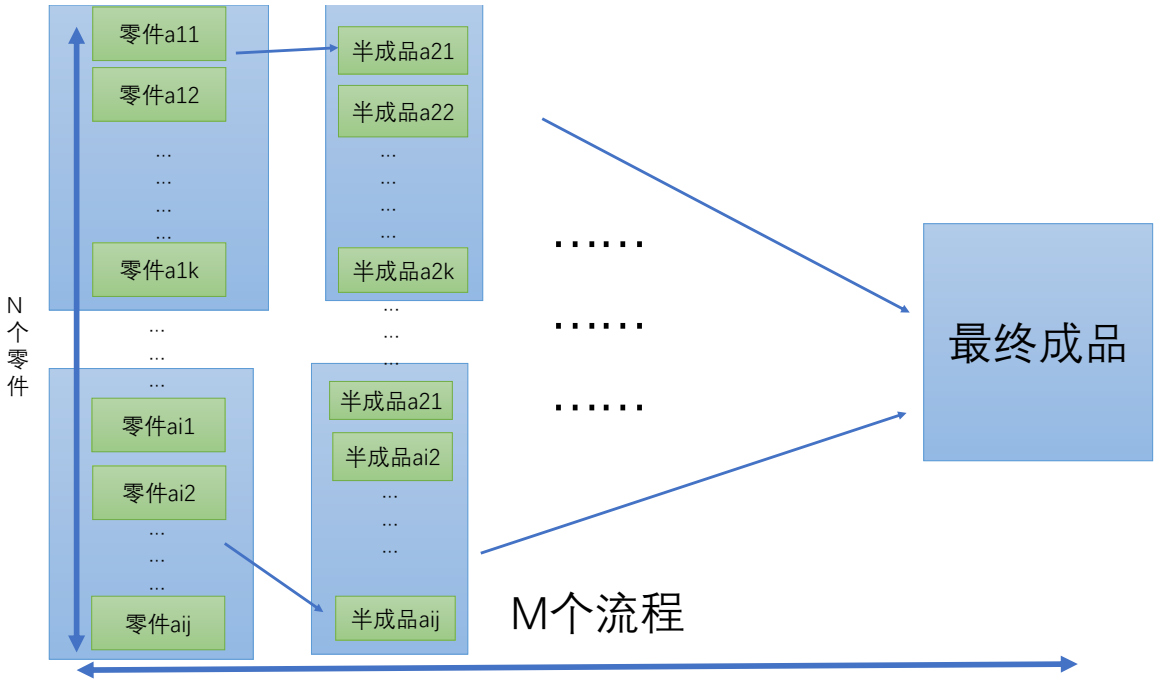


图 6　m,n 种情况的示意图

Step3：模型的推广

上述步骤中，我们讨论了 8 个零件，2 道工序的情况，对于更为普遍的情况，我们同样可以利用该方法进行求解。如问题三中提到的 n 个零件，m 道工序，我们同样可以将其分为多个模块，每个模块的讨论同问题二，使用贪心算法，以每道工序的局部最优解的组合为所得最优解，再利用遗传算法进行优化，优化程度通过控制变异的范围进行控制。

7.2 求解结果

对问题三中的具体情况进行求解，我们得出最优解为：检测零件一、二、四、五、七、八，半成品一、二、三和成品。此外，还有一些较优解，在此用表格进行表示：

是否检测	零件 1	零件 2	零件 3	零件 4	零件 5	零件 6	零件 7	零件 8
最优解	是	是	否	是	是	否	是	是
较优解 1	否	是	否	否	是	否	是	是
较优解 2	是	是	否	是	是	否	是	否
较优解 3	否	是	否	否	是	否	是	否

是否检测	半成品 1	半成品 2	半成品 3	成品	是否拆解	半成品 1	半成品 2	半成品 3	成品	利润
最优解	是	是	是	是		是	是	是	是	11516.2
较优解 1	是	是	是	是		是	是	是	是	11452.1
较优解 2	是	是	是	是		是	是	是	是	9467.7
较优解 3	是	是	是	是		是	是	是	是	9403.6

可以看到，几种较优解仅在零件的检测方面有差异。另一方面，我们一开始使用贪心算法的得到的结果为较优解 2，对比优化后的局部最优解可以看出，通过使用遗传算法进行优化，得到结果的利润有明显提高。

八、 问题四的模型的建立和求解

8.1 模型建立

对于问题四，我们的最终目标在于尽可能的接近正确的次品率。由于我们事先并不知道准确的次品率，因此需要对次品率有预先的估值，然后对其进行检验。具体流程见图：

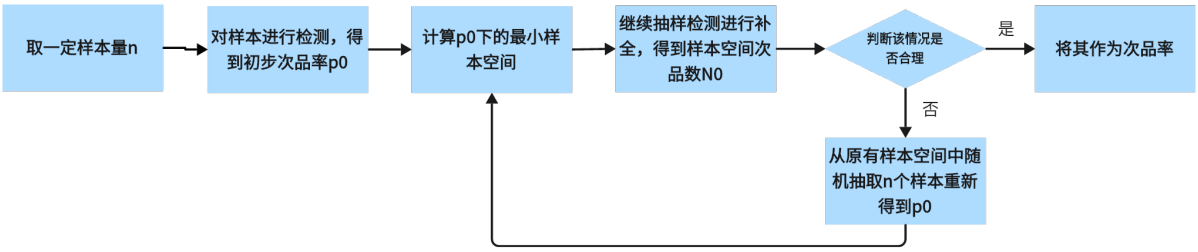


图 7　次品率的确认流程

8.2 模型求解

8.2.1 初步次品率的确定

我们首先给出零件中次品率的假设。方式为先检测一个较小量的样本数 n（如 50），计算出次品率, 将其作为初步次品率 p_0。

$$p_0 = \frac{\bar{n}}{n}$$

8.2.2 假设检验

根据初步确定的次品率值，我们做出零假设 H_0

H_0: 得到的初步次品率值是可靠的

接下来需要检验该假设的可靠性。在此我们采用统计学中假设检验的方式，判断零假设在信度为 95% 情况下是否为真。

我们以该估计值作为概率计算最小可靠样本空间 N，根据 N 的数量，额外抽取 N-n 个样本进行检验。为使模型处理不同数据的标准一致，此处我们规定在 95% 的信度下对次品率值进行估计，误差范围 E 仍然取 5%。利用 Cochrane 公式，得到最小的样本空间。从已经抽到的样本空间出发，补全样本空间 N，使数量达到最低限度。此处有

$$N = \frac{1.96p_0(1-p_0)}{0.05^2}$$

接下来进行信度为 95% 的假设性检验，判断总体样本中的次品数目 $\bar{N}$ 是否处于可接受范围内，即：

$$\bar{N} \leq Np_0 + Z\alpha\sqrt{Np_0(1-p_0)}$$

在此 $Z\alpha$ 取值同为 1.96。如果检验合格，我们可以认为估计的次品率可信度达到了 95%，因此接受零假设，将次品率带入我们第二问和第三问的模型中求解最佳策略。这里重复使用了问题二和问题三的决策模型。

如果我们验证的结果是拒绝零假设，那么从新加入的样本空间中抽取同样量的个体（如 50），重新计算得到的次品率。再次应用 Cochrane 公式并根据公式结果补充样本，重复验证零假设的正确性，直到零假设成立位置。

值得注意的是，我们所剔除的数据都是一些异常数据，因此得到的最终结果一定会逼近正确的次品率。采用这种方法得到的策略一般来说误差不会特别大，会落在可接受的范围内。因此这种做法是相对合理的。

8.3 求解结果

通过上述步骤，我们可以得到一个较为准确的次品率，进而可以使用问题二和问题三中的模型进行求解，得到一个接近于全局最优解的结果，从而实现我们对最优决策的求解。通过重复使用前两个问题中的数学模型，我们可以发现这些模型结果是相对正确的。

九、 灵敏度分析

在所有模型中，问题二中的模型是解决决策问题的基础，我们因此对问题二进行灵敏度分析，改变零件的次品率，观察决策的变化。

<table>
<tr><td colspan="5">灵敏度分析 1</td><td colspan="3">灵敏度分析 3</td></tr>
<tr><td rowspan="2">情况</td><td colspan="3">零配件 1</td><td rowspan="2">最优策略编号</td><td colspan="2">成品</td><td rowspan="2">最优策略编号</td></tr>
<tr><td>次品率</td><td>购买单价</td><td>检测成本</td><td>调换损失</td><td>拆解费用</td></tr>
<tr><td>1</td><td>10%</td><td rowspan="4">4</td><td rowspan="4">2</td><td>8</td><td></td><td rowspan="6">5</td><td>8</td></tr>
<tr><td>2</td><td>50%</td><td>8</td><td>10</td><td>8</td></tr>
<tr><td>3</td><td>70%</td><td>8</td><td>14</td><td>8</td></tr>
<tr><td>4</td><td>90%</td><td>7</td><td>18</td><td>10</td></tr>
<tr><td>5</td><td></td><td></td><td></td><td></td><td>26</td><td>10</td></tr>
<tr><td>6</td><td></td><td></td><td></td><td></td><td>38</td><td>10</td></tr>
</table>

<table>
<tr><td colspan="5">灵敏度分析 2</td><td colspan="4">灵敏度分析 4</td></tr>
<tr><td rowspan="2">情况</td><td colspan="3">零配件 1</td><td rowspan="2">最优策略编号</td><td colspan="3">零配件 1</td><td rowspan="2">最优策略编号</td></tr>
<tr><td>次品率</td><td>购买单价</td><td>检测成本</td><td>次品率</td><td>购买单价</td><td>检测成本</td></tr>
<tr><td>1</td><td rowspan="4">10%</td><td rowspan="4">4</td><td>2</td><td>8</td><td>10%</td><td rowspan="6">4</td><td rowspan="6">10</td><td>1</td></tr>
<tr><td>2</td><td>10</td><td>4</td><td>20%</td><td>1</td></tr>
<tr><td>3</td><td>14</td><td>4</td><td>30%</td><td>1</td></tr>
<tr><td>4</td><td>18</td><td>4</td><td>40%</td><td>2</td></tr>
<tr><td>5</td><td></td><td></td><td></td><td>60%</td><td>2</td></tr>
<tr><td>6</td><td></td><td></td><td></td><td>80%</td><td>2</td></tr>
</table>

策略编号	1	2	3	4	5	6	7	8	9	10	11	12	13	14
x_1 到 x_4 的取值	0000	0010	0100	0101	0110	0111	1000	1001	1010	1011	1100	1101	1110	1111

由图可知，我们的模型对零配件次品率（分析 1）、检测成本（分析 2）、调换损失（分析 3）、以及极端情况下的求解（分析 4）都保持较好的稳定度，既没有因为变量的轻微变动而剧烈地改变应对策略，也没有在变量大幅变化时保持同一个策略。

十、 模型的评价

10.1 模型的优点

- 优点 1 在计算成本时，考虑到了不同决策在每个流程中的影响，便于分步解决问题。
- 优点 2 模型实现的逻辑较为清晰，利于理解和调整。
- 优点 3 模型可行性强，决策效果好，对于降低生产成本，提升商品利润有较好的适用度。

10.2 模型的缺点

- 缺点 1 贪心算法得到的局部最优解不一定是全局最优解，因此模型仍有优化空间。
- 缺点 2 问题四中的检验方法可能需要一定成本，虽然可靠但效率可能不是最高的。

参考文献

[1] 司守奎, 孙玺菁. 数学建模算法与应用[M]. 北京: 国防工业出版社, 2011.

[2] 斑马. 假设检验——这一篇文章就够了[EB/OL]. 2021. https://www.zhihu.com/tardis/zm/art/86178674?source_id=1005,Last accessed on 2024-9-8.

附录 A　文件列表

文件名	功能描述
question1.py	问题一程序代码
question2.m	问题二程序代码
question3_1a	问题三程序代码一
question3_2b	问题三程序代码二
question3_3a	问题三程序代码三

附录 B　代码

question1.py

```python
import numpy as np

# 求最小样本空间

# 公司预估零件良品率的最小值
good = 0.9
# 不良品率的最大值
wrong = 0.1

# 取标准正态分布的单侧临界值位置的值，这里取0.05,因此za/2为z0.025,查表知实际值为1.96
key = 1.96

# 预估的方差np(1-p) 二项分布

# 误差空间：e
e = 0.05


# 带入cochrane公式
n = np.square(key)*good*wrong/np.square(e)
print("样本空间的最小值为:")
print(n)
```

```python

N = 139
print("因此n取139.")
# 下一步对N中的元素做次品最大值讨论
# 第一小问
# 查表正态分布的单侧临界值位置的值，这里我们取a为0.05，此时za = 1.645
key2 = 1.645

# 方差
theta = np.sqrt(N*good*wrong)

max_wrong = round(wrong * N + theta*key2)

print("在95%%的置信度下，当次品数量超过%d是我们认为零假设是不正确的，即次品率超过了10%%"%max_wrong)

# 第二小问
# 查表正态分布上单侧临界值位置的值，当a = 0.1时，可知大小za =1.28
key3 = 1.28
N2 = np.square(key2)*good*wrong / np.square(e)

print("N2:",N2)
max_wrong2 = wrong * N2 + theta*key3

print("在90%%的信度下，当次品数量少于%d是我们认为零假设是正确的，接受次品，即次品率不会超过10%%"%max_wrong2)
```

question2.m

```
%初始化数据
optim=0;
optimdecision=[];
allsolution=[];
alldecision=[];
p1=[0.1,0.2,0.1,0.2,0.1,0.05]; %零件1的次品率
p2=[0.1,0.2,0.1,0.2,0.2,0.05]; %零件2的次品率
p3=[0.1,0.2,0.1,0.2,0.1,0.05]; %成品的次品率
n1=100; %零件1的数量
n2=100; %零件2的数量
buy1=4;   %零件1的购买单价
buy2=18;  %零件2的购买单价
assemble=6; %成品的装配成本
exchange=[6,6,30,30,10,10]; %调换损失
test1=[2,2,2,1,8,2]; %零件1的检测成本
test2=[3,3,3,1,1,3];
test3=[3,3,3,2,2,3]; %成品的检测成本
dismantle=[5,5,5,5,5,40]; %拆解费用
price=56;  %市场售价

%运算
for x1=0:1
    for x2=0:1
        for x3=0:1
            for x4=0:1
                n3=min((1-x1.*p1).*n1,(1-x2.*p2).*n2);
                max_product=min(n1.*(1-p1),n2.*(1-p2));
                p=1-(1-(1-x1).*p1).*(1-(1-x2).*p2).*(1-p3);
                z1=n1.*buy1+n2.*buy2+x1.*n1.*test1+x2.*n2.*test2;
                z2=n3.*assemble+x3.*n3.*test3;
                z3=x4.*((n3.*p/(1-p)).*(dismantle+assemble+x1.*test1+x2.*test2));
                z4=(1-x3).*n3.*p.*exchange;
```

```
                tem=(n3.*(1-p)).^(1-x4).*(max_product.^x4).*price-z1-z2-z3-z4;
                allsolution=[allsolution;tem];
                alldecision=[alldecision;x1,x2,x3,x4];
            end
        end
    end
end

rows_to_remove = (alldecision(:,1)==0) &(alldecision(:,2)==0) & (alldecision(:,4)==1);% 找到所有后两位等于0 1的行
alldecision(rows_to_remove,:) = [];% 删除这些行
allsolution(rows_to_remove,:)=[]; %删除所有结果中不符合的
[optim,index]=max(allsolution,[],1); %对每一列取最优
optimdecision=alldecision(index,:);
disp(optimdecision);
x=1:6;
plot(x,allsolution(1:end,:),'LineWidth',3);
xlabel("情况");
ylabel("总利润");
legend("策略1","策略2","策略3","策略4","策略5","策略6","策略7","策略8","策略9","策略10","策略11","策略12",'Location','southwest');
title("不同情况和策略下的总利润比较")
```

question3_1a.m

问题三中三个零件组装成一件半成品的情形

```
%初始化数据
allsolution=[];
alldecision=[];

p1=0.1; %零件1的次品率
p2=0.1; %零件2的次品率
p3=0.1; %零件3的次品率
p4=0.1; %成品的次品率
n1=100; %零件1的数量
n2=100; %零件2的数量
n3=100; %零件3的数量
%n4为成品的数量
buy1=2;    %零件1的购买单价
buy2=8;   %零件2的购买单价
buy3=12; %零件3的购买单价
assemble=8; %成品的装配成本
exchange=40/3; %调换损失
test1=1; %零件1的检测成本
test2=1;
test3=2;
test4=4; %成品的检测成本
dismantle=6; %拆解费用
price=200/3;   %市场售价

%运算
for x1=0:1
    for x2=0:1
        for x3=0:1
            for y1=0:1
                for y2=0:1
                    n4=min([(1-x1.*p1).*n1,(1-x2.*p2).*n2,(1-x3.*p3).*n3]);
                    max_product=min([n1.*(1-p1),n2.*(1-p2),n3
```

```
.*(1-p3)]);
                    p=1-(1-(1-x1).*p1).*(1-(1-x2).*p2).*(1-(1-x3).*p3).*(1-p4); %成品中次品的占比
                    z1=n1.*buy1+n2.*buy2+n3.*buy3+x1.*n1.*test1+x2.*n2.*test2+x3.*n3.*test3;
                    z2=n4.*assemble+y1.*n4.*test4;
                    z3=y2.*((n4.*p/(1-p)).*(dismantle+assemble+x1.*test1+x2.*test2+x3.*n3.*test3));
                    z4=(1-y1).*n4.*p.*exchange;
                    tem=(n4.*(1-p)).^(1-y2).*(max_product.^y2).*price-z1-z2-z3-z4;
                    allsolution=[allsolution;tem];
                    alldecision=[alldecision;x1,x2,x3,y1,y2];
                end
            end
        end
    end
end

rows_to_remove = (alldecision(:,4)==0) & (alldecision(:,5)==1);% 找到所有后两位等于0 1的行
alldecision(rows_to_remove,:) = [];% 删除这些行
allsolution(rows_to_remove,:)=[]; %删除所有结果中不符合的
[tem,index]=sort(allsolution,'descend'); % 降序排序
optimdecision=alldecision(index(1:2),:);
disp(tem(1:2));
disp(optimdecision);
```

question3_2b.m

问题三中两个零件组装成一件半成品的情形

```
%初始化数据
optim=0;
optimdecision=[];
allsolution=[];
alldecision=[];
p1=0.1; %零件1的次品率
p2=0.1; %零件2的次品率
p3=0.1; %成品的次品率
n1=100; %零件1的数量
n2=100; %零件2的数量
buy1=8;   %零件1的购买单价
buy2=12;  %零件2的购买单价
assemble=8; %成品的装配成本
exchange=40/3; %调换损失
test1=1; %零件1的检测成本
test2=2;
test3=4; %成品的检测成本
dismantle=6; %拆解费用
price=200/3;  %市场售价

%运算
for x1=0:1
    for x2=0:1
        for x3=0:1
            for x4=0:1
                n3=min((1-x1.*p1).*n1,(1-x2.*p2).*n2);
                max_product=min(n1.*(1-p1),n2.*(1-p2));
                p=1-(1-(1-x1).*p1).*(1-(1-x2).*p2).*(1-p3);
                z1=n1.*buy1+n2.*buy2+x1.*n1.*test1+x2.*n2.*test2;
                z2=n3.*assemble+x3.*n3.*test3;
                z3=x4.*((n3.*p/(1-p)).*(dismantle+assemble+x1.*test1+x2.*test2));
```

```
                z4=(1-x3).*n3.*p.*exchange;
                tem=(n3.*(1-p)).^(1-x4).*(max_product.^x4).*price-z1-z2-z3-z4;
                allsolution=[allsolution;tem];
                alldecision=[alldecision;x1,x2,x3,x4];
            end
        end
    end
end

rows_to_remove = (alldecision(:,3)==0) & (alldecision(:,4)==1);% 找到所有后两位等于0 1的行
alldecision(rows_to_remove,:) = [];% 删除这些行
allsolution(rows_to_remove,:)=[]; %删除所有结果中不符合的
[tem,index]=sort(allsolution,'descend'); % 降序排序
optimdecision=alldecision(index(1:2),:);
disp(tem);
disp(optimdecision);
```

question3_3a.m

问题三中三件半成品组装成一件成品的情形

```
%初始化数据
allsolution=[];
alldecision=[];

x11=0;x21=1;x31=0;
fp1=1-(1-(1-x11).*0.1).*(1-(1-x21).*0.1).*(1-(1-x31).*0.1)
   .*(1-0.1);
p1=0; %零件1的次品率
p2=0; %零件2的次品率

x12=1;x22=0;
fp2=1-(1-(1-x12).*0.1).*(1-(1-x22).*0.1).*(1-0.1);
p3=fp2; %零件3的次品率

p4=0.1; %成品的次品率
n1=100; %零件1的数量
n2=100; %零件2的数量
n3=100; %零件3的数量
%n4为成品的数量
buy1=2+8+12+8;    %零件1的购买单价
buy2=2+8+12+8;   %零件2的购买单价
buy3=8+12+8; %零件3的购买单价
assemble=8; %成品的装配成本
exchange=40; %调换损失
test1=4; %零件1的检测成本
test2=4;
test3=4;
test4=6; %成品的检测成本
dismantle=10; %拆解费用
price=200;  %市场售价

%运算
for x1=0:1
```

```
    for x2=0:1
        for x3=0:1
            for y1=0:1
                for y2=0:1
                    n4=min([(1-x1.*p1).*n1,(1-x2.*p2).*n2,(1-x3.*p3).*n3]);
                    max_product=min([n1.*(1-p1),n2.*(1-p2),n3.*(1-p3)]);
                    p=1-(1-(1-x1).*p1).*(1-(1-x2).*p2).*(1-(1-x3).*p3).*(1-p4); %成品中次品的占比
                    z1=n1.*buy1+n2.*buy2+n3.*buy3+x1.*n1.*test1+x2.*n2.*test2+x3.*n3.*test3;
                    z2=n4.*assemble+y1.*n4.*test4;
                    z3=y2.*((n4.*p/(1-p)).*(dismantle+assemble+x1.*test1+x2.*test2+x3.*n3.*test3));
                    z4=(1-y1).*n4.*p.*exchange;
                    tem=(n4.*(1-p)).^(1-y2).*(max_product.^y2).*price-z1-z2-z3-z4;
                    allsolution=[allsolution;tem];
                    alldecision=[alldecision;x1,x2,x3,y1,y2];
                end
            end
        end
    end
end

rows_to_remove = ((alldecision(:,4)==0) & (alldecision(:,5)==1))|((alldecision(:,1)==0)&(alldecision(:,2)==0)&(alldecision(:,3)==0)&(alldecision(:,4)==1)&(alldecision(:,5)==1));% 找到所有后两位等于0 1的行
alldecision(rows_to_remove,:) = [];% 删除这些行
allsolution(rows_to_remove,:)=[]; %删除所有结果中不符合的
[tem,index]=sort(allsolution,'descend'); %对每一列取最优


```

```
disp(tem(1));
```